Check for updates

Received: 6 July 2022 | Revised: 20 February 2023 | Accepted: 3 March 2023

IET Intelligent Transport Systems

DOI: 10.1049/itr2.12354

The Institution of Engineering and Technology WILEY

ORIGINAL RESEARCH

CenLight: Centralized traffic grid signal optimization via action and state decomposition

Jia Wu[1] | Yunchuan Ran[2] | Yican Lou[1] | Lingzhou Shu[1]

[1]School of Information and Software, University of Electronic Science and Technology of China, Chengdu, China

[2]School of Engineering, Rice University, Houston, Texas, USA

Correspondence
Jia Wu, School of Information and Software, University of Electronic Science and Technology of China, Chengdu, China.
Email: jiawu@uestc.edu.cn

Funding information
National Natural Science Foundation of China, Grant/Award Number: 61503059

Abstract

The centralized traffic grid signal control by the reinforcement learning method is challenging due to the difficulties of searching policy in the large state and action space. In order to solve these problems, a deep reinforcement learning (DRL) method via the action and state decomposition mechanism is proposed. We apply long short-term memory to construct the agent which decomposes the high-dimensional state and action space into sub-spaces and makes decisions incrementally. This is a significant difference between our method and other methods. Through the specifically designed structure of the agent, the difficulty of searching policies can be mitigated, and our method can effectively control the traffic lights in a grid with hundreds of intersections. Experiments on synthetic data and real-world data show that our method has better performance than traditional control methods and state-of-the-art DRL-based methods with an improvement of 21% on the queue length on synthetic data and the best travel time with an improvement of 9.35% on real-world data.

1 | INTRODUCTION

Traffic congestion is a major problem in modern society. Traffic congestion not only causes huge economic losses but also increases traffic accidents [1]. An effective solution is to adjust the traffic flow at intersections through traffic lights (signals). So, how to plan traffic signals efficiently and flexibly has become a hot topic for many researchers.

Early methods used preset traffic signal [2] or planned timing based on historical traffic flow data [3] to control the traffic. These kinds of methods are easy to implement, but cannot dynamically adapt to the traffic environment. Besides, the efficiency of control relies heavily on hand-crafted rules. Therefore, the control efficiency is quite low due to the influence of various factors that have not been considered in the dynamic traffic environment.

To overcome this problem, new methods that can adapt to traffic state have emerged [4], where the duration of each phase is dynamically adjusted according to the current traffic flow. Although these methods greatly improve the control efficiency, the adaptive timing can only be adjusted according to artificially preset strategies and under certain assumptions, which often do not hold in the real environment.

Reinforcement learning (RL) is a promising approach for optimizing traffic lights. Since RL is a learning framework that allows the agent to learn the optimal control policy based on feedback from the environment, RL has great potential to deal with issues that cannot be solved by conventional techniques. Many RL-based solutions are developed, such as references[5–7], where the RL agents observe the state of the traffic, and then control traffic lights to reduce traffic congestion or to achieve other criteria, such as reducing the average delay, the average queue length, and the average waiting time. RL-based solutions differ in terms of definitions of the state, the action and the reward function, the structure of the agent, and the training method.

The early RL-based methods establish a Q table composed of traffic states and actions and update the Q table through Q-learning, Sarsa, or other algorithms, such as in references[8–10], which focus on single intersection control. References[8] and [9] compare their methods with the traditional traffic control method, fixed-time control. Reference[10] compares its method

Jia Wu and Yunchuan Ran contributed equally to this work.

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.
© 2023 The Authors. *IET Intelligent Transport Systems* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

with the fixed-time control and the actuated control. Researches ([11, 12]) further extend to multiple intersections with model-based RL. However, traditional RL solutions are only suitable for discrete and low-dimensional state space.

The emergence and development of deep reinforcement learning (DRL) provide a new way to solve the aforementioned challenges. By introducing a deep neural network to approximate high-dimensional state space [13], DRL can learn informative representations of the environment and effectively explore the state-action space. Recently, a large number of researches of DRL-based traffic control have emerged [14–16].

Despite the significant success achieved by the above-mentioned researches, they still face several challenges. The centralized control with a single agent has successfully controlled the traffic for one intersection or multiple intersections. However, as the scale of the network grows, the large state-action space makes it difficult for the agent to learn. To deal with the curse of dimensionality, decentralized methods with multi-agents are employed, where each agent controls a single intersection and shares information with the adjacent intersections. The decentralized methods can be applied in a large-scale network, but the cooperation of the agents is quite challenging since each agent makes a decision by considering the close agents to achieve the globally optimal policy. Authors in reference[17] have pointed out that the training process of the agents may suffer from instability.

Here, we aim to *explore the possibility of the centralized method in the large-scale grid* since the centralized control has more advantages than the decentralized one in the following aspects: (1) Centralized methods use single agent to control the whole grid. The agent outputs the joint action for all traffic signals. Theoretically, owing to global observability, the centralized methods can find the global optimal solution from the perspective of global control. While the decentralized methods are easy to fall into the local optimal solutions due to the limited communications. (2) As the scale of the grid increases, the communication between agents requires a large number of computational resources. (3) Due to the complexity of a multi-agent environment, the convergence of agents is hard to achieve.

To mitigate the curse of dimensionality problem, we propose a centralized method called *CenLight* based on the decomposition of the state and action space. We specially design a new structure of the agent which makes decisions incrementally by using long short-term memory (LSTM) network to decompose the joint state-action space, thus the search space is reduced at each decision step. Besides, we propose a new training method for the decomposition mechanism.

Our contributions are as follows:

1. **Propose a new way to mitigate the curse of dimension**: By the special design of the structure, the agent can decompose the joint state-action space into sub-space and searches the policy only in sub-spaces, so that the agent can tackle the large-scale traffic network.
2. **Learn association between actions by the recurrent network**: Although early researchers have introduced the recurrent network into the RL agent, they only use it to learn the temporal relationship between states [18], and do not apply it to decompose the high-dimensional state-action space. As far as we know, we are the first to employ it to decompose the action space of traffic control problem and coordinate the sub-components of the joint action to learn the globally optimal policy.
3. **Propose a new way to train the agent**: Based on the decomposition mechanism, we propose a method to improve the training speed and stability without sacrificing performance. Besides, we rectify the target calculation for the value function to make our method suitable for traffic control problems
4. **Extensive experiments on both synthetic and real-world datasets**: We conduct experiments on **four** different datasets including both synthetic and real-world datasets to demonstrate the effectiveness of our model. We tackle a large-scale net with about 200 intersections.

The remainder of this paper is organized as follows. Section 2 first formulates the traffic control problem in the framework of RL and then presents the related work of traffic control based on RL. Section 3 formualtes the traffic conrol problem in the RL framework. Section 4 details our method, including the design and the workflow of our agent, the training algorithm, and the designs of state, action, and reward. Section 5 presents the experiment results and analysis. Finally, the paper is concluded in Section 6.

2 | RELATED WORK

2.1 | Preliminary

RL is an approach in which an agent interacts with the environment through learning strategies to maximize the cumulative rewards. An RL problem is often modelled as a Markov decision process (MDP) with five key elements $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ [19]:

1. $\mathcal{S}$ represents a set of environmental and agent states;
2. $\mathcal{A}$ represents a set of actions of the agent;
3. $\mathcal{P}$ denotes the probability of transition from the current state s to the next state s' after taking the action a, that is, $P_a(s, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$;
4. $\mathcal{R}$ is reward for a transition from current state s to next state s' after taking the action a, that is, $R_a(s, s')$.
5. γ is a discount factor for future rewards.

An RL agent works as follows: at each time t, it perceives the state of traffic environment s_t, and then selects an action a_t. The environment transfers to s_{t+1} according to the current s_t and a_t, and emits the reward r_t to the agent. After a long time interaction (learning), the agent obtains a sequence of states, actions, and rewards: $\tau = \{s_1, a_1, r_1, \dots, s_t, a_t, r_t, \dots\}$, which is called a trajectory.

A policy π is the agent's behaviour, which maps from the state to the action. The agent's goal is to find the optimal policy π that maximizes the reward it receives over time. More precisely, the goal of the agent is to maximize the expectation of

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

the cumulative reward: $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, where γ is the discount factor, $\gamma \in (0, 1]$. A larger γ means the agent's policy has a higher dependency on future rewards, while a smaller γ leads the agent to care more about the current reward. G_t is also called the return.

The traffic control process can be modelled as an MDP decision-making process, in which the agent controls the traffic light, and the whole traffic network is the environment. The agent observes the state of the traffic environment, and selects an appropriate traffic light control action. By continuously interacting with the environment, the agent learns a policy π to maximize the cumulative rewards. The reward can be the average queue length, the average speed, etc.

2.2 | Traffic control based on RL

The DRL-based traffic control methods can be classified into two groups according to the traffic control mode: centralized control and decentralized control [20]. The centralized control methods usually have one RL agent which learns the optimal policy for the whole grid. The decentralized control methods have multiple RL agents cooperating to find the optimal policy.

Several researchers have applied centralized control (single agent) for traffic control ([21, 22]). However, the centralized control methods suffer from the curse of dimension, that is, the search space of policy increases exponentially as the scale of the network grows. Some researchers apply the divide-and-conquer method to overcome this problem. Reference[21] proposes a hierarchical method to reduce the search space of policy, which is tested in a 3×3 grid. Another research [22] uses a different way to address the problem of large search space and the experiment is implemented in a 2×2 grid. Authors in reference[22] considered the dedicated short-range communication (DSRC) technology for vehicle-to-infrastructure (V2I) communication. The agent controls traffic signals by partial detection of vehicles at intersections, that is, the RL agent only observes the vehicles with DSRC. The proposed method is verified in a 4×4 grid. As far as we know, the centralized method is applied only in a small-scale grid due to the curse of dimensionality.

The decentralized control methods can easily handle the large-scale network by multi-agent approaches. Each agent controls one or a small part of the signals and cooperates with other agents in adjacent regions. Since each agent observes the local state of the grid, multi-agent methods require collaboration to achieve a global optimal control. The current research of multi-agent can be divided into the following four categories: max-plus-based collaboration, graph-based approach, hierarchical control, and information sharing:

- **Max-plus-based collaboration**: The max-plus method is to exchange information between adjacent agents so that the optimal strategy can be determined within a limited number of iterations. Reference [23] proposes the first deep-RL-based multi-intersection control mechanism, which defines a special reward function based on a variety of traffic conditions, such as accidents or jams, emergency stops, traffic light changes, and vehicle waiting practices. In order to coordinate the signals, reference[23] uses a transfer planning technique for a smaller set of intersections and links the learning results to a larger set of intersections with the max-plus coordination algorithm. Reference[24] is similar to reference [23], which also uses the max-plus algorithm to coordinate signals, but it adds migration learning and uses recurrent neural network (RNN) network instead of a fully connected neural network to predict Q value. The final results show that the average delay reduces by using the RNN network.
- **Graph-based approach**: In the work of reference[25], a graph convolutional network (GCN) is used to learn the representation of multiple intersections. GCN is combined with a specific RL algorithm called k-step neural fitted Q-iteration [26], which updates the agent in a distributed manner. Reference[27] employs a bidirectional triple graph to represent the road network. CRRank sorting algorithm [28] is used to sort the edges according to the specified scores, and then the key intersections are identified according to the key edges. The key intersections are optimized to improve learning efficiency. The proposed method is tested in grids with 20, 50, and 100 intersections on simulation of urban mobility (SUMO), compared with fixed-time, SOTL [3], Q-learning, and DQN controllers. Reference[29] proposes a multi-agent method called CoLight, which uses a graph attention network (GAT) to obtain the influence of its neighbours. CoLight compares with other state-of-the-art methods in both synthetic and real-world datasets. The results show that CoLight has a strong performance over other methods. Reference[30] proposes a variant of CoLight, which considers the uncertainty of the environment and designs a new state based on transportation theory.
- **Hierarchical control**: For the hierarchical control method, the control mode is generally divided into two parts: the bottom layer and the top layer. The bottom layer is responsible for controlling the signals in a single or block area, and the top layer is responsible for coordinating the bottom layers. For example, reference[31] optimizes regional traffic through intersection control at the lower level and optimizes overall traffic by adjusting the degree of optimization of regional traffic at the top level. Multi-intersection learning is achieved based on thresholds collected from a single intersection, rather than by setting phase cycles or phase duration. The action set of the higher-level controller is to increase or decrease the threshold and to change the sensitivity of each intersection to the adjacent intersections.
- **Information sharing**: There are three kinds of methods for information sharing: sharing rewards, sharing Q values, and sharing traffic information. In reference[32], only the rewards are shared between adjacent agents, and the states of adjacent agents are not considered when making decisions.

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Reference[33] proposes a method of Q value transfer, where the agent considers the influence of the latest actions of its adjacencies in the process of policy learning. Specially, the optimal Q values of the neighbour agents at the latest time step are transferred to the loss function of the Q network. Reference [34] adopts the independent advantage actor critic (A2C) method of multiple agents. In order to solve the stability problem, it introduces fingerprints of neighbour intersections to provide each agent with information regarding the local policies and traffic distribution. Besides, it employs a spatial discount factor which allows each agent to focus on improving the local traffic. Reference[35] introduces a new traffic movement representation with a pressure of queuing and demand of running for vehicles to improve the performance. Reference[36] presents a new concept " intensity," which ensures that the reward design and state representation can accurately reflect the status of vehicles.

In addition to the above-mentioned works, recent research, such as reference [37] uses two game theory-aided RL algorithms leveraging Nash equilibrium and RL to achieve cooperation. References[38, 39] explore the possibility of real-world implementation of RL-based control methods; reference[40] proposes a novel context-aware multi-agent broad reinforcement learning approach based on broad reinforcement learning for mixed pedestrian–vehicle adaptive traffic light control. Although the multi-agent control methods have achieved great success in specific scenarios, they still face several challenges. For example, as the scale of the network structure increases, the required resources for computation increases dramatically, such as max-plus and graph-based methods. Hierarchical control is an ideal method, but how to carry out efficient and orderly cooperation between the top and the bottom is a big challenge. Too much interference from the top to the bottom may affect the learning efficiency and the stability at the bottom; while too little interference may lead to difficulty in achieving the global optimum. For the information sharing methods, they have the issues of information loss, for example, the Q value in reference[33] cannot fully embed the information contained in the intersection.

We believe the centralized control method can overcome the shortcomings of multi-agent if the curse of dimensionality can be mitigated. Here, we aim to explore the limits and potential of the centralized approach. Towards that, we propose a decomposition mechanism to reduce the dimension of the action and state space. In this new mechanism, we design a new structure of the agent-based recurrent network to explore the interaction and relationship between actions. Thus, the cooperation of intersections can be achieved.

3 | PROBLEM FORMULATION

Here, we aim to design a centralized deep RL-based control method with *one single* agent to control K intersections (suppose that each intersection has one traffic signal). We formulate the traffic signal control problem as an MDP with five elements tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

3.1 | State space $\mathcal{S}$

Agent observes the global state of environment at time step t, $\boldsymbol{s}_t \in \mathcal{S}$, and $\boldsymbol{s}_t = \{s_t^1, s_t^2, \dots, s_t^K\}$, where K represents the number of all intersections and each component s_t^k consists of critical information of an intersection k.

Generally, the state must include key traffic information. With the development of vehicle communication technology (V2I and V2X), road cameras, and faster Internet connection technology (5G), a comprehensive observation of road vehicles with a low delay has become a reality. We can make a more comprehensive design of the state, which indicates the information on the road network observed by the agent. Our design of the state includes the following information:

1. The number of vehicles presented on each lane of the intersection;
2. The current signal phase.
3. The duration of the phase denotes how long the current phase has not changed until time t.

3.2 | Action space $\mathcal{A}$

$\boldsymbol{a}_t = \{a_t^1, \dots, a_t^K\} \in \mathcal{A}$ represents the joint action for the whole network, where a_t^k represents the decision for intersection k. The agent selects, every Δt period of time, a phase from its predefined phase set to be activated for each intersection.

It is noted that if each signal has four phases, the action space is 4^K, which grows exponentially with K. Here, we devised a decomposition mechanism to mitigate the curse of dimensionality problem.

3.3 | Transition probability

$\mathcal{P}$ denotes the probability transition matrix. Giving the current state $\boldsymbol{s}_t$ and the action $\boldsymbol{a}_t$ of the agent at time step t, $\mathcal{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t, \boldsymbol{a}_t) : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ represents the probability of arriving at the state $\boldsymbol{s}_{t+1}$. Generally, it is impossible to get the probability transition matrix in traffic control problem due to the complex and changeable environment.

3.4 | Reward $\mathcal{R}$

The agent receives the reward $\boldsymbol{r}_t \in \mathcal{R}$ from the environment after making a decision action $\boldsymbol{a}_t$, where $\boldsymbol{r}_t = \{r_t^1, \dots, r_t^K\}$ and each element r_t^k represents the the negative value of the sum of the queue lengths on the approaching lanes of intersection k at time t, $k \in [1, K]$.

To optimize the road traffic flow, the goal of the agent is to maximize the cumulative reward, which is computed by:

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

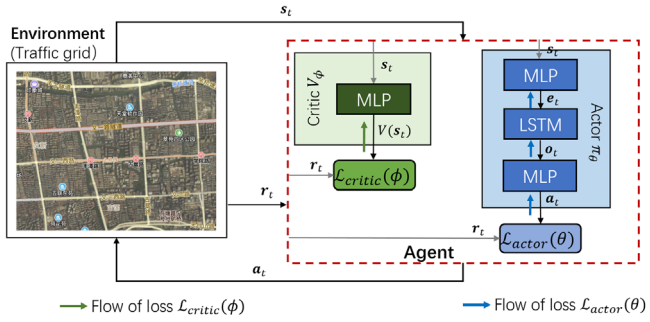


FIGURE 1 Internal structure of the agent. The agent consists of an actor model π_θ (a policy learner) and a critic model V_ϕ (a value function). The critic model receives the state $\boldsymbol{s}_t$ and outputs the decision $\boldsymbol{a}_t$. The critic model estimates the state of the agent following the current policy. Two models are optimized from the loss defined in Equations (8) and (10).

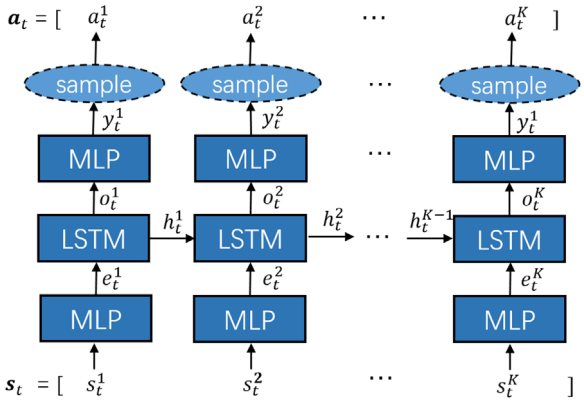


FIGURE 2 Workflow of the actor model. For each time t, the actor unrolls K times to give a joint action $\boldsymbol{a}_t = \{a_t^1, \dots, a_t^K\}$ (K is the number of intersections to control). At each unrolling step k, it receives the state of one intersection s_t^k and makes a decision a_t^k based on the previous actions $a_t^1, \dots, a_t^{k-1}$.

$R(t) = \sum_{t'=t}^{T} \gamma^{t'-t} \boldsymbol{r}_{t'-t}$, where T is the length of an episode and γ is the discount factor for the future reward.

4 | OUR METHOD

4.1 | Design of the agent

The internal structure of our agent is described in Figure 1, which is composed of two models: an actor model π_θ (a policy learner) and a critic model V_ϕ (a value function). The actor model makes decisions conditioned on the current state while the critic model approximates the value function, which estimates the future rewards that start in the current state and follow the policy. These two models interact to shift the policy towards more profitable states. In this part, we will first introduce the actor model about its state-action space decomposition mechanism to alleviate the curse of dimensionality problem. Then, we will introduce the critic model. Next, we will present the training algorithm. Finally, we will summarize the overall workflow of the agent.

• **Actor model**

The challenge of centralized control is the large search space of policy, that is, the curse of dimension in joint action space. Assume that there are K intersections presented in the traffic grid. The agent decides, for each intersection, whether to keep the current phase or switch to another phase. The dimension of joint action space is 2^K, which is extremely large for the large K value. Even though there are many designs of action that can alleviate the problem, they all have some limitations. For example, the traffic control agent in reference[21] decides when and how to coordinate the local controllers. Thus, the action space decreases from 2^K to $K+1$. But this action design sacrifices the flexibility of selecting actions.

Generally, a hard problem can be solved effectively by dividing it into multiple sub-problems that are easier to handle. The solutions to the sub-problems are then combined to give a solution to the original problem. Since the joint action space is very large, it is difficult for the agent to make a joint action $\boldsymbol{a}$ for all intersections, where $\boldsymbol{a} = \{a^1, \dots, a^K\}$. However, if the agent makes the joint action incrementally (sequentially), that is, it makes one decision a^k for an intersection at each step, based on previous decisions $a^1, \dots, a^{k-1}$, the action space can be greatly reduced at each step and it is much easier for the agent to search a good joint action $\boldsymbol{a}$. As authors in reference[41] claimed: any complex high-dimensional action can be selected incrementally, component by component, where each component probability also depends on components already selected earlier.

Therefore, we consider traffic control as a sequential decision problem. We can easily infer that such a design of the control requires the agent to remember previous information. Hence, the core part of the agent should remember the past and make decisions based on what it has learned from the past. Recurrent neural networks have proved to be effective for processing sequential data by remembering past information. We use the LSTM neural network to construct the core part of the agent (see Figure 2) because LSTM can learn the conditionality among the data. Compared with RNN, LSTM can prevent the neural network output for a given input from either decaying or exploding as it cycles through the feedback loops.

The actor model works as follows (as shown in Figure 2): at each simulation step t, it receives the state $\boldsymbol{s}_t$, then unrolls K times, where K denotes the total number of intersections, to output the joint action $\boldsymbol{a}_t$ for the grid.

At each unrolling step k ($k \in [1, K]$), the state of the intersection s_t^k is fed to a multi-layer perceptron (MLP), which embeds the state into a more expressible feature vector $\boldsymbol{e}_t = \{e_t^1, \dots, e_t^K\}$:

$$e_t^k = Embed\left(s_t^k\right) = ReLu\left(s_t^k W_e + b_e\right) \tag{1}$$

where W_e and b_e are the weight matrix and deviation vector of the embedding layer to be learned, respectively.

Then, $\boldsymbol{e}_t$ is fed to the core part of the agent, that is, LSTM layer, which helps the agent explore the correlation between

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

sub-actions:

$$\begin{aligned} f_t^k &= \sigma\left(W_f \cdot \left[h_t^{k-1}, s_t^k\right] + b_f\right) \\ i_t^k &= \sigma\left(W_i \cdot \left[h_t^{k-1}, s_t^k\right] + b_i\right) \\ \tilde{C}_t^k &= tanh\left(W_C \cdot \left[h_t^{k-1}, s_t^k\right] + b_C\right) \\ C_t^k &= f_t^k * C_t^{k-1} + i_t^k * \tilde{C}_t^k \\ o_t^k &= \sigma\left(W_o \cdot \left[h_t^{k-1}, s_t^k\right] + b_o\right) \end{aligned} \tag{2}$$

where W_f and b_f are parameters of the forget gate layer; W_i and b_i are parameters of the input gate layer; W_C and b_C are parameters of the cell state; W_o and b_o denote the parameters for the output.

Next, the output of LSTM layer is fed a projection layer (MLP layer) to convert the output of the LSTM layer to a low-dimensional representation, which represents the probabilities of the candidate actions for the signal at the intersection k. The agent samples the control decision a_t^k based on y_t^k:

$$\begin{aligned} y_t^k &= softmax(W_p \cdot o_t^k + b_p) \\ a_t^k &= sample(y_t^k) \end{aligned} \tag{3}$$

The hidden information at the unrolling step k is updated and fed to the agent at the next unrolling step $k+1$.

$$h_t^k = o_t^k * tanh(C_t^k) \tag{4}$$

After K steps, the agent outputs the joint control decisions $\boldsymbol{a}_t = \{a_t^1, \ldots, a_t^K\}$ for K intersections (signals). Naturally, the search space at each unrolling step decreases sharply, thus it is much easier for the agent to search the policy.

It is noted that the order of outputting the decision for intersection does not affect the final results. We will prove that in the following lemma.

Lemma 1. *The order of outputting the decision for the intersection does not affect the final results.*

Proof. Let $P(A_1 A_2 \ldots A_K)$ denote the probability of making decisions for K signals in the grid, where A_k represents the random variable of making a decision for the signal k. According to our method, the agent makes decisions one by one, and makes a decision based on the previous decisions, $P(A_1 A_2 \ldots A_K)$ can be written based on the conditional probability:

$$\begin{aligned} &P(A_1 A_2 \ldots A_K) \\ &= P(A_1)P(A_2|A_1)P(A_3|A_1 A_2) \ldots P(A_K|A_1 A_2 \ldots A_{K-1}) \\ &= P(A_2)P(A_3|A_2)P(A_4|A_2 A_3) \ldots P(A_1|A_2 A_3 \ldots A_K) \\ &= P(A_4)P(A_2|A_4)P(A_7|A_4 A_2) \ldots P(A_9|A_1 A_2 \ldots A_K) \\ &= \ldots \end{aligned} \tag{5}$$

We can easily see that the agent outputs decisions incrementally, that is, the agent makes a decision based on the previous decisions due to the memory of its core part, thus the order in which it makes decisions for intersections will not affect the final result. □

In Section 5.3.2, we will conduct an experiment to demonstrate that.

• **Critic model**

As shown in Figure 2, the agent is composed of two models: *actor* for computing an action based on a state and *critic* to evaluate the value of the state. The actor controls the behaviours of the agent by learning the optimal policy (policy based). The critic, on the other hand, evaluates the state by computing the value function (value based), which cooperates with the actor model to efficiently learn the policy. The critic model works as follows: at each time step t, it is given the state of the grid $\boldsymbol{s}_t = \{s_t^1, \ldots, s_t^K\}$, and outputs the estimated values $V_\phi(\boldsymbol{s}_t) = \{V_\phi(s_t^1), \ldots, V_\phi(s_t^K)\}$.

$$V_\phi(s_t^k) = ReLu(W_c \cdot s_t^k + b_c) \tag{6}$$

where $\phi = \{W_c, b_c\}$ are parameters of the critic models. The critic model is an MLP network. The reasons we use a simple neural network to design a critic model are the following: (a) the critic model does not need to learn the relationship between the values of the states at intersections; (b) considering time efficiency, we want the network structure to be as simple as possible.

4.2 | Agent training algorithm

We use one of the modern Actor-Critic algorithms (AC), the Proximal Policy Optimization (PPO-Clip) algorithm to train the agent. AC algorithms take advantage of value-based and policy-based RL methods.

PPO-Clip algorithm [42] is a flexible and effective RL algorithm, which performs more stable than the traditional ones. Specifically, the PPO-Clip algorithm uses two tricks to improve training performance: importance sampling and KL divergence to constrain the gradient update.

Since the agent of AC algorithms is composed of two models: *actor* π_θ and *critic* V_ϕ, we train them separately. Due to the special design of the agent, we propose a new way to calculate the advantage and the loss function during the training procedure.

First, given the state $\boldsymbol{s}_t = \{s_t^1, \ldots, s_t^K\}$, the action of the agent $\boldsymbol{a}_t = \{a_t^1, \ldots, a_t^K\}$ and the reward it receives $\boldsymbol{r}_t = \{r_t^1, \ldots, r_t^K\}$, we adopt the generalized advantage estimator [43] to estimate the discounted advantage function $A(s_t^k, a_t^k)$, which is calculated as follows:

$$\begin{aligned} \delta_t^k &= r_t^k + \gamma V_\phi(s_{t+1}^k) - V_\phi(s_t^k) \\ A\left(s_t^k, a_t^k\right) &= \sum_{t'=t}^{T} (\gamma\lambda)^{t'-t} \delta_{t'}^k \end{aligned} \tag{7}$$

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

where $V_\phi(\cdot)$ is the output of the critic model with parameters ϕ; λ and γ are discounted factors which adjust bias-variance tradeoff, $0 \le \lambda \le 1$ and $0 \le \gamma \le 1$.

Then, we calculate the loss function of the actor model:

$$\mathcal{L}_{\text{actor}} = \frac{1}{K \times T} \sum_{t=1}^{T} \sum_{k=1}^{K} \min \left(pr_t^k(\theta) A\left(s_t^k, a_t^k\right), \right.$$
$$\left. \text{clip}\left(pr_t^k(\theta), 1-\epsilon, 1+\epsilon\right) A\left(s_t^k, a_t^k\right) \right)$$
$$pr_t^k(\theta) = \frac{\pi_\theta\left(a_t^k \mid s_t^k\right)}{\pi_{\theta_{\text{old}}}\left(a_t^k \mid s_t^k\right)} \tag{8}$$

where $\pi_\theta(\cdot)$ is the actor with the current parameters θ and $\pi_{\theta_{\text{old}}}$ denotes the actor model with the old parameters. $pr_t^k(\theta)$ denote the probability ratio between old and new policies. The motivation of *clip* function in (8) has been discussed in reference[42]: it keeps the new policy close to the old policy by clipping $pr_t(\theta)$ outside of the interval $[1-\epsilon, 1+\epsilon]$. In (8), we calculate $pr_t^k(\theta)$, the ratio of actions for the signal k, which are made by the new and the old policies and clip it into $[1-\epsilon, 1+\epsilon]$, where $\epsilon = 0.2$. The parameters of the actor model are updated by a gradient ascend algorithm:

$$\theta \leftarrow \theta + \nabla_\theta \mathcal{L}_{\text{actor}}(\theta) \tag{9}$$

The critic model $V_\phi(\cdot)$ approximates the value function of the most recent policy. The critic model is updated by minimizing the mean-squared-error objective:

$$\mathcal{L}_{\text{critic}} = \frac{1}{K \times T} \sum_{t=1}^{T} \sum_{k=1}^{K} (G_t^k - V(s_t^k))^2 \tag{10}$$

where G_t^k is the reward-to-go for the agent at kth unrolling step in simulation time t.

Generally, $G_t^k \doteq \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}^k$. It should be noted that traffic control problem is different from playing Atari games. For Atari games, the game will end in the finite horizon; while for traffic control problem, the control continues, that is, it has infinite horizon. Since we train the agent in a finite horizon, T, the conventional calculation of G_t does not take into account the future return. To prevent the large variance of the target of the value function, we design a new way to give reward-to-go as follows:

$$G_t^k = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}^k + \gamma^{T-t+1} V_\phi(s_{T+1}^k) \tag{11}$$

where $V_\phi(s_{T+1}^k)$ denotes the estimated value of the state at $T+1$, which indicates the estimation of marginal impact of actor on the future return, thus making G_t^k change smoothly. The parameters of the critic model is updated by a gradient descent algorithm:

$$\phi \leftarrow \phi - \nabla_\phi \mathcal{L}_{\text{critic}}(\phi) \tag{12}$$

ALGORITHM 1 Agent Training Algorithm

Input: initial policy parameters θ; initial value function parameters ϕ; N, M, and B represent numbers of iterations to train the agent, the actor model, and the critic model of the agent, respectively
for $i \in \{1, \dots, N\}$ **do**
  for $t \in \{1, \dots, T\}$ **do**
    Actor π_θ receives the state $\boldsymbol{s}_t$, and outputs the decision $\boldsymbol{a}_t$ based on Equations (1), (2) and (3)
    Collect transitions $\{\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{r}_t\}$ into $\boldsymbol{D}_i$
  end for
  Estimate advantages $A(\boldsymbol{s}_t, \boldsymbol{a}_t)$ through Equation (7)
  Estimate reward function $\boldsymbol{G}_t$ through Equation (11)
  $\pi_{\theta_{old}} \leftarrow \pi_\theta$
  for $j \in \{1, \dots, M\}$ **do**
    Calculate $\mathcal{L}_{actor}$ through Equation (8)
    Update π_θ by a gradient ascend method
  end for
  for $j \in \{1, \dots, B\}$ **do**
    Calculate $\mathcal{L}_{critic}$ through Equation (10)
    Update V_ϕ by a gradient descend method
  end for
end for

4.3 | Workflow of the method

Our method is detailed in Algorithm 1. At each iteration, the actor π_θ collects T time-steps data. The actor receives, at each t, the state of the grid $\boldsymbol{s}_t$, and makes decisions $\boldsymbol{a}_t$ for the grid based on Equations (1) and (3). Then, we update the actor and the critic models using the collected data. The above procedure repeats N interactions to obtain the optimized policy.

5 | EXPERIMENT SETUP

We designed a series of experiments to investigate the following questions:

1. How much can our method improve compared with state-of-the-art DRL methods and traditional controls? Can the proposed method be adapted to different traffic networks with different modes of inflow rates?
2. Can the proposed method quickly converge during training? And is the training process stable?
3. Does the introduction of the decomposition mechanism help to improve the search procedure?

The setups of the experiment are listed as follows:

1. **Simulation environment**: Our experiments are tested in two simulation platforms: SUMO for synthetic data and CityFlow for real-world data. The reasons why we chose two

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

simulation platforms are: (1) SUMO is a traditional traffic simulation platform and it is easier to compare traditional traffic control methods under it; (2) CityFlow is a newly designed open-source traffic simulator, which runs much faster for RL-based control method. Several state-of-the-art DRL-based methods, for example, references[29, 44], use CityFlow as the simulator to test their algorithms.

- **SUMO:** SUMO[1] is an open-source traffic simulation tool. Through Traci interface that it provides, RL agent can interact with the simulated traffic environment. In the experiments on synthetic data, each intersection is a four-way intersection in four directions, and each direction has two lanes (an approaching lane and a leaving lane). The inflow road is 200 m long and the inner road is 100 m long. The road speed limit is 35 km per hour. Each intersection is controlled by a traffic signal with two phases: the north–south phase and the east–west phase. A yellow light is displayed for the clearance interval, which lasts 3 s, and the signal phase cannot be switched during the yellow light. During the simulation, the agent observes the environment and makes decisions every 3 s (the action interval). Each episode of the simulation is 10 min.
- **CityFlow**: CityFlow[2] is an open-source traffic simulator that supports flexible definitions for the road network and traffic flow, and is especially suitable for large-scale RL-based traffic signal control. We compare our method with state-of-the-art DRL-based methods for traffic signal control on real-world datasets. The setting of datasets is consistent with CoLight [29], such as the same traffic flow settings, each intersection has four phases, and the action interval is 10 s. Each episode of the simulation is 1 h.

2. **Agent structure and training detail**: The actor is composed of three layers of neural networks, one layer of embedding (64 neural units), one layer of LSTM (64 neural units), and one output layer. The output layer uses Softmax function for activation. The critic is composed of two layers of MLP. The first layer has 100 neural units, and the second layer is the output layer with 1 neural unit. ReLu is used as the activation function for MLP layers in the actor and the critic models. For the experiments on synthetic datasets, the learning rate $\alpha = 0.005$, the discount factor $\gamma = 0.8$, $\lambda = 1$; for the experiments on real-world datasets, the learning rate $\alpha = 0.002$, the discount factor $\gamma = 0.9$, $\lambda = 1$ (other DRL-based methods use the same parameters). The reason why we use two different sets of parameters is to highlight the difference between our method and other methods in different situations. Especially in the ablation experiment, we use a smaller learning rate α and a larger discount factor γ to reduce the training speed and highlight the change of reward in each episode.
3. **Evaluation metric**: The evaluation indicators are the mean vehicle queue length, the mean vehicle speed, and the average travel time. The mean queue length is defined as the cumulative number of vehicles stopped before the conflict zone divided by the simulation time. The mean vehicle speed is the average speed of all vehicles presented in the network over a period of time. The average travel time evaluates the average travel time of all vehicles spent in the grid.

All experiments are implemented on the computer with the following configuration: Intel(R) Core(TM) i7-6800k processor; NVIDIA TITAN Xp.

5.1 | Performance comparison

The following experiments aim to verify whether our method can outperform the state-of-the-art DRL-based methods and traditional traffic control methods. We conducted experiments on real-world data and synthetic data, respectively.

5.1.1 | Compared with DRL methods

We compared our method with the state-of-the-art DRL methods on CityFlow using real datasets from the public dataset.[3] We choose the traffic data from three cities (see Figure 3): Jinan, Hangzhou, and New York [29, 44, 45]. Jinan is a 3×4 traffic network with about a total flowrate 6300 vehicles per hour; Hangzhou is a 4×4 traffic network with about a total flowrate 3000 vehicles per hour; New York is a much bigger traffic network which has 196 intersections (28×7), and the total flowrate is about 11500 vehicles per hour. The datasets and traffic flow settings are the same as CoLight [29]. In order to be consistent with the mainstream methods, we use the average travel time as the evaluation metric.

Baselines

Variants of our methods:

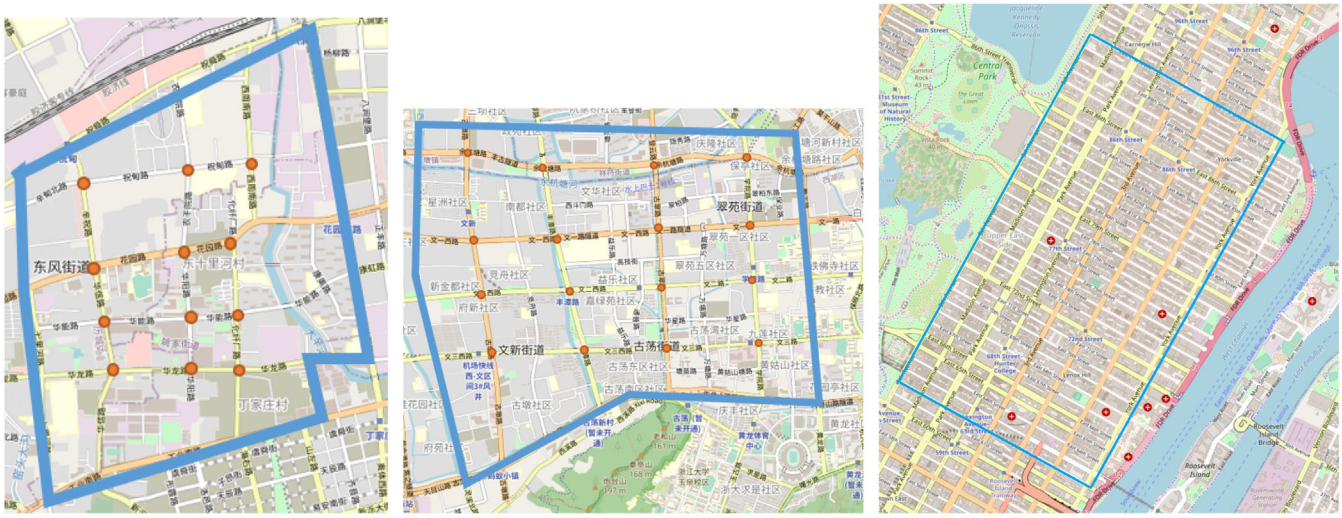
- *Bi-CenLight*: a variant of CenLight, which uses a bidirectional LSTM (Bi-LSTM) network, an extension of LSTMs that processes the data in both forward and backward direction to improve cooperation between signals.
- *RNN-CenLight*: a variant of CenLight, which uses an RNN network as the core part, to compare with Bi-CenLight and CenLight.

Centralized control methods:

- *HRC* [21]: a centralized traffic control method, which has two layers of controllers for hierarchical regional control. The traffic signals are controlled by the local controllers based on well-adjusted policies. The master controller coordinates the local controllers based on the RL algorithm.

[1] http://sumo.sourceforge.net/
[2] https://github.com/cityflow-project/CityFlow/
[3] https://traffic-signal-control.github.io/

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



(a) Dongfeng sub-district 3 × 4 , Jinan, China (b) Gudang sub-district 4×4, Hangzhou, China (c) Manhattan 28 × 7, New York City, USA

FIGURE 3 Road networks for real-world datasets.

Decentralized control methods:

- *CoLight* [29]: a multi-agent RL-based method, which cooperates different agents using GAT. GAT allows to learn the influence of neighbours for the target intersection.
- *Individual RL* [46]: a multi-agent method which has no cooperation between agents. Agents do not share parameters and each agent updates its own parameter based on the samples collected at the intersection controlled by itself.
- *OneModel* [34]: a multi-agent method which is similar with Individual RL. The difference between these two methods are: "fingerprints" and rewards of neighbouring agents are included to achieve cooperation between agents; besides, agents of OneModel share parameters.
- *Neighbour RL* [47]: similar to OneModel, agents share their parameters. The state of this method includes much more information than OneModel, which concatenates the state of the agent with the neighbouring intersections' information.

Empirical results

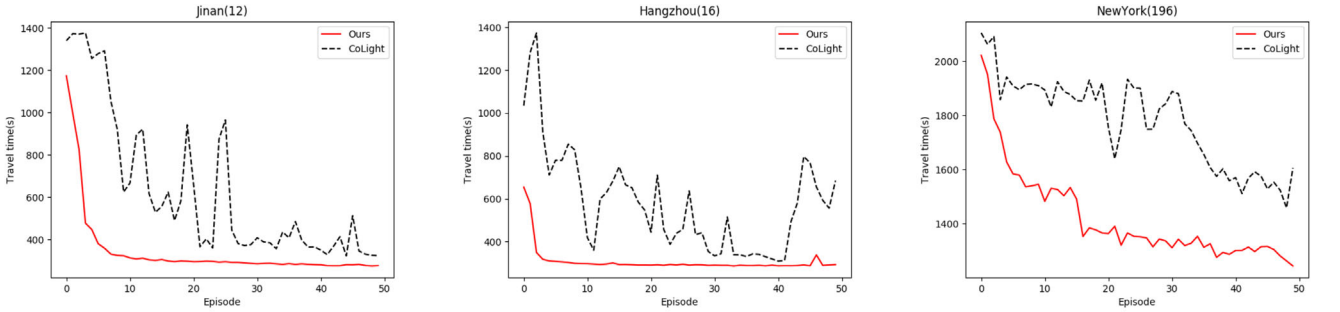
Table 1 shows the comparison between ours and other DRL methods on real-world data. Figure 4 presents the learning curves between ours and CoLight. We have the following observations: (1) Our method has consistent performance on real-world data, with at least 4.28% improvement over CoLight. We consider this advantage comes from the centralized control method, which can learn the optimal policy by observing the entire network. (2) The variants of our method: RNN-CenLight performs the worst; Bi-CenLight has a slight advantage over CenLight since Bi-LSTM can better observe the conditionality in the sequence than unidirectional LSTM. However, the computational cost of Bi-LSTM is higher. We prefer to use CenLight since it balances the computational cost and the performance. (3) The centralized control method HRC performs the worst since the local control policy is too simple, that is, fixed-cycle time; the master controller can fine-tune only one local controller at each action interval, which is not flexible enough. (4) OneModel performs better than Individual RL on most datasets since OneModel achieves cooperation between agents. (5) The performance of Neighbour RL is the worst among the decentralized methods since it directly concatenates the neighbouring information without considering the importance of different neighbours. (6) CoLight significantly outperforms other baselines since it uses GAT with an attention mechanism to differentiate the importance of neighbours. However, our method converges much faster than CoLight due to the special design of our agent, which is simpler and more flexible (see Figure 4).

TABLE 1 Comparison between different DRL methods w.r.t. the average travel time on real-world data.

Model	Jinan	Hangzhou	New York
CenLight	270.65	285.68	1354.24
Bi-CenLight	**270.19**	**284.81**	**1303.92**
RNN-CenLight	277.03	290.00	1408.37
HRC [21]	420.06	470.08	/
CoLight [29]*	290.57	297.53	1438.49
Individual RL [46]	341.29	355.12	/
OneModel [34]	316.23	413.24	1890.72
Neighbour RL [47]	425.24	616.33	1952.87
Improvement	7.01%	4.28%	9.35%

*We use the same datasets and traffic flow settings as CoLight [29]. The improvement is obtained by comparison with CoLight.

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



(a) Dongfeng sub-district 3 × 4 , Jinan, China (b) Gudang sub-district 4×4, Hangzhou, China (c) Manhattan 28 × 7, New York City, USA

FIGURE 4 Travel time curves between ours and CoLight [29] on three real-world datasets.

TABLE 2 Details of fixed-time control and actuated control.

	Phase		
Control model	**North–south**	**East–west**	**Duration (s)**
Fixed-time control	Green	Red	20
	Yellow	Yellow	3
	Red	Green	20
	Yellow	Yellow	3
Actuated control	Green	Red	16–24
	Yellow	Yellow	3
	Red	Green	16–24
	Yellow	Yellow	3

5.1.2 | Compared with traditional traffic control methods

We compared our method with two traditional methods in four different traffic grids: 1×3, 2×2, 2×3, and 3×3, with two different modes of releasing the traffic flow:

- *Mode 1 – flows with a fixed number of vehicles*: each vehicle will be given a random departure time which is equidistributed within the time interval of the flow.
- *Mode 2 – flows with a random number of vehicles*: a vehicle will be emitted into the grid randomly with the given probability.

Apparently, mode 2 makes the traffic environment more complex and challenging by randomly emitting vehicles.

Baselines

Traditional control methods [48] (see Table 2 for detailed settings) includes:

- *Fixed-time control*: the duration of each phase remains unchanged.
- *Actuated control*: an adaptive control method that can dynamically extend or shorten the duration of the current traffic light phase according to the flowrate, but the duration of the phase must be limited.

Empirical results

Table 3 summarizes the performance of different methods under mode 1, from which we can see that our method performs best in all scenarios, and shows a great advantage compared with other methods.

Table 4 summarizes the performance of different methods under mode 2, from which we can easily observe that the performance of all methods degrades compared with that under mode 1 since vehicles are emitted randomly and the scenarios become more complex in mode 2. However, our method still outperforms other methods.

5.2 | Robustness of training process

The following experiments are implemented under various networks 1×3, 2×2, 2×3, and 3×3 traffic grids with different inflow rates of vehicles on SUMO. Each experiment runs three times. The experiments aim to verify the following questions:

1. Whether the agent can learn effectively during the training process?
2. Can the training process converge stably?

Figure 5 presents the learning curves of the agent in different grids under different flowrates, from which we can easily observe that: (1) the learning curves of agents at low flowrate rise and converge quickly, which indicates that the agent can learn fast in this scenario (Figure 5a); (2) The learning curves fluctuate largely and the agent needs more training steps to converge as the complexity of the traffic grid grows (Figure 5b). However, the cumulative rewards still rise steadily and eventually converge.

5.3 | Ablations

Here, we ablate the features of our approach to verify the effectiveness of our method. The experiments are conducted on synthetic datasets.

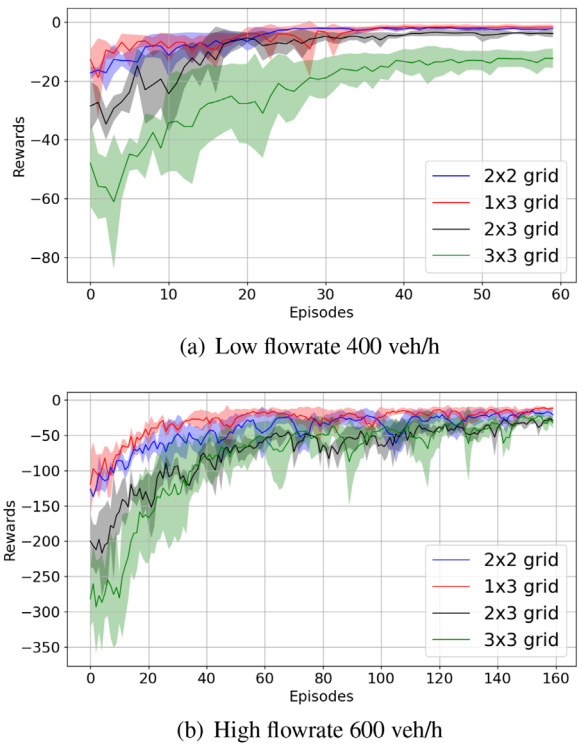
17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 3 Performance of all methods on synthetic data under mode 1 (flows with a fixed number of vehicles). Each lane has 400 and 600 vehicles per hour.

		1×3		2×2		2×3		3×3	
Metric	**Control model**	**400**	**600**	**400**	**600**	**400**	**600**	**400**	**600**
	Ours	**0.36 ± 0.21**	**4.92 ± 0.42**	**0.48 ± 0.20**	**5.10 ± 0.20**	**1.84 ± 0.24**	**5.91 ± 0.67**	**2.42 ± 0.32**	**6.71 ± 1.10**
Mean	Fixed time	7.99 ± 0.53	14.54 ± 0.34	12.62 ± 0.39	26.54 ± 0.45	19.84 ± 0.49	36.90 ± 0.46	30.10 ± 0.55	58.27 ± 0.60
Queue	Actuated	3.91 ± 0.52	9.47 ± 0.43	11.95 ± 0.54	23.73 ± 0.36	6.01 ± 1.06	20.34 ± 0.29	9.69 ± 0.82	22.58 ± 0.63
	Ours	**19.34 ± 0.12**	**16.65 ± 0.24**	**20.93 ± 0.54**	**17.71 ± 0.20**	**19.45 ± 0.21**	**18.36 ± 0.37**	**20.02 ± 0.66**	**18.89 ± 0.45**
Mean	Fixed time	14.88 ± 0.36	13.53 ± 0.37	14.15 ± 0.31	11.89 ± 0.22	13.81 ± 0.24	12.22 ± 0.20	13.49 ± 0.23	11.86 ± 0.14
speed	Actuated	16.97 ± 0.35	14.96 ± 0.27	14.73 ± 0.40	13.50 ± 0.23	15.13 ± 0.48	13.22 ± 0.14	16.87 ± 0.19	15.40 ± 0.13

TABLE 4 Performance of all methods on synthetic data under mode 2 (flows of probability-based vehicle flowrate). Each lane has 0.11 and 0.17 veh/s.

		1×3		2×2		2×3		3×3	
Metric	**Control model**	**0.11**	**0.17**	**0.11**	**0.17**	**0.11**	**0.17**	**0.11**	**0.17**
	Ours	**2.76 ± 1.06**	**6.88 ± 1.52**	**3.28 ± 0.29**	**13.86 ± 1.93**	**3.38 ± 0.32**	**19.17 ± 1.30**	**6.12 ± 0.51**	**24.09 ± 4.55**
Mean	Fixed time	7.90 ± 0.23	17.09 ± 1.05	10.93 ± 0.84	24.87 ± 1.65	19.10 ± 1.68	39.81 ± 2.75	30.22 ± 2.15	62.10 ± 2.85
Queue	Actuated	4.35 ± 0.08	14.30 ± 0.79	4.81 ± 0.79	20.55 ± 2.34	15.56 ± 2.02	30.87 ± 3.60	11.39 ± 1.93	35.96 ± 3.01
	Ours	**18.33 ± 0.70**	**14.32 ± 0.49**	**17.99 ± 0.25**	**14.08 ± 0.49**	**19.50 ± 0.14**	**14.36 ± 0.28**	**18.44 ± 0.24**	**14.56 ± 0.73**
Mean	Fixed time	14.83 ± 0.41	12.94 ± 0.33	14.63 ± 0.21	12.33 ± 0.20	13.65 ± 0.28	11.80 ± 0.21	13.23 ± 0.11	11.48 ± 0.10
speed	Actuated	16.50 ± 0.19	13.69 ± 0.21	16.65 ± 0.16	13.02 ± 0.40	14.28 ± 0.33	13.38 ± 0.28	16.23 ± 0.29	13.55 ± 0.39



(a) Low flowrate 400 veh/h

(b) High flowrate 600 veh/h

FIGURE 5 Learning curves of training our method under different flowrates. The solid line and the shade show the average reward and the standard deviation over three independent runs, respectively.

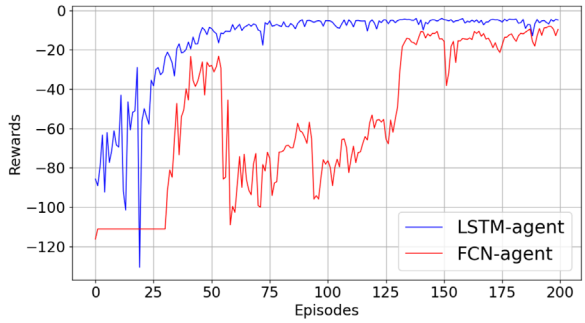
5.3.1 | Action decomposition mechanism

This part aims to verify whether the action decomposition mechanism indeed improves the performance. That is, the LSTM-based actor can search the policy more easily and coordinate the control of signals. We compared our method to an RL method similar to our method, except that the actor is composed of a fully connected network (FCN). We refer to this RL-based method as FCN-agent.

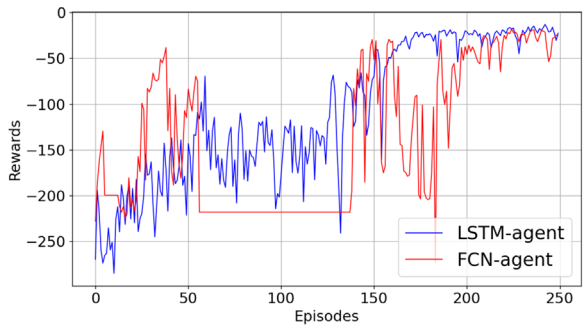
The FCN-agent works as follows: At each time t, it receives s_t^k of intersection k and outputs the decision a_t^k for the signal k. The procedure repeats K times and gives decisions $\boldsymbol{a}_t = \{a_t^1, \ldots, a_t^K\}$ for the grid. The main difference between our agent and FCN-agent is that our agent can learn the relation between decisions through LSTM neural network, and the joint actions $\boldsymbol{a}_t$ cooperate with each other; while the FCN-agent outputs independent action without considering the interaction with each other. The training methods, critic model, and the corresponding parameter settings for FCN-agent are the same as our method.

Figure 6 presents the learning curves of our method and FCN-agent in different types of traffic grids. It is easy to see that the training process of our method (LSTM-agent) is much more stable and the learning curve of the FCN-agent fluctuates heavily. We consider the main reason is the mutual influence between intersections. FCN-agent cannot consider the interaction between other actions (intersections). While LSTM-agent can explore the inner relation due to the intrinsic

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



(a) 1 × 3 gird under flowrate 600 veh/h



(b) 2 × 3 grid under flowrate 600 veh/h

FIGURE 6 Learning curves of our method (LSTM-agent) and FCN-agent. FCN-agent is similar with our model, except the actor model of FCN-agent is a fully connected net.

TABLE 5 Comparison between LSTM-agent and FCN-agent in different grids. The results are averaged across three random seeds.

Scenario	Control model	Flow rate	Mean queue	Mean speed
1×3	LSTM-agent	600	**4.56 ± 0.83**	**17.27 ± 0.31**
	FCN-agent		9.65 ± 0.35	14.59 ± 0.26
2×2	LSTM-agent	600	**6.95 ± 0.61**	**16.88 ± 0.46**
	FCN-agent		9.21 ± 0.92	15.41 ± 0.51
2×3	LSTM-agent	600	**13.68 ± 2.18**	**15.58 ± 0.64**
	FCN-agent		23.62 ± 2.03	13.03 ± 0.28
3×3	LSTM-agent	600	**13.10 ± 2.01**	**17.09 ± 0.58**
	FCN-agent		22.41 ± 1.33	14.93 ± 0.20

characteristics of the LSTM structure, hence the training process becomes more stable and less wavy.

Table 5 summarizes the average speed and queue length collected in different grids under 600 vehicles per hour. We can see the LSTM-agent outperforms the FCN-agent in all criteria.

5.3.2 | Different orders of action sequences

Here, we aim to test whether the order in which the agent outputs the traffic signals of intersections will affect the overall control efficiency. We implemented the experiments in a 2 × 3 grid under the flowrate 600 vehicles per hour. We compared the four different orders as shown in Table 6.

TABLE 6 Different orders of action sequence (S_i denotes signal i).

Sequence 1: $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \rightarrow S_5 \rightarrow S_6$
Sequence 2: $S_1 \rightarrow S_4 \rightarrow S_2 \rightarrow S_5 \rightarrow S_3 \rightarrow S_6$
Sequence 3: $S_2 \rightarrow S_1 \rightarrow S_3 \rightarrow S_5 \rightarrow S_4 \rightarrow S_6$
Sequence 4: $S_6 \rightarrow S_5 \rightarrow S_4 \rightarrow S_3 \rightarrow S_2 \rightarrow S_1$

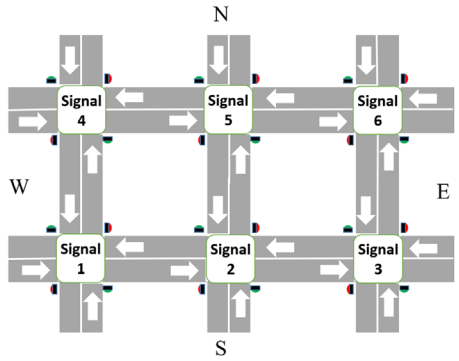


FIGURE 7 Positions of traffic control signal in a 2×3 grid.

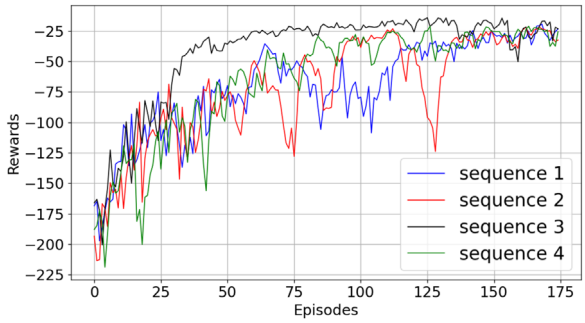


FIGURE 8 Learning curves of training agents with different orders of action sequence in 2 × 3 grid under flowrate 600 vehicles per hour. The sequences are defined in Table 6.

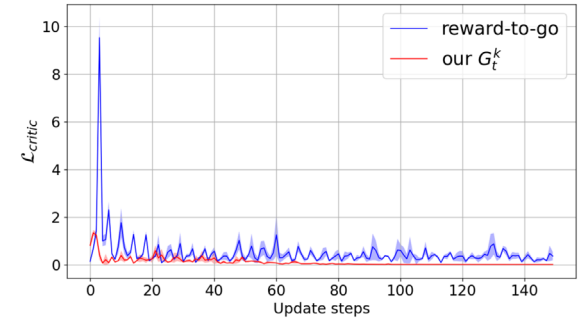
The labels of all traffic signal positions in traffic grid are shown in Figure 7. For example, "sequence 2" in Table 6 denotes that during the six unrolling steps, the agent outputs at first the decision a_t^1 for signal 1, then based on the decision a_t^1, outputs the decision a_t^2 for signal 4, at the third step for signal 2, …, at last for signal 6 based on the decisions of previous five steps $\boldsymbol{a}_t^{1:5}$. In the following experiments, we will verify whether the order will affect the final performance.

Figure 8 presents the learning curves of rewards with different action sequences, from which we can see that: (1) the learning curve of sequence 3 rises faster and stably than other sequences. (2) Sequence 2 is the most unstable one. (3) Even if there are subtle differences in the training processes, the final values of reward are very close. (4) The experimental result suggests that different orders of action sequences will not affect the final results. Table 7 summarizes the details of the mean queue length and the average speed of different orders

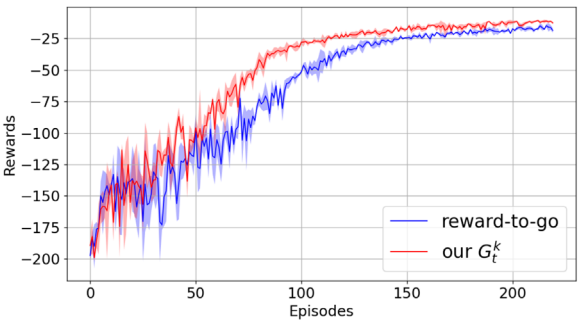
17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 7 Mean queue and mean speed of different sequences in 2× 3 grid under 600 vehicles per hour. The results are averaged across three random seeds.

Sequence	Mean queue	Mean speed
Sequence 1	13.68 ± 2.18	15.58 ± 0.64
Sequence 2	13.34 ± 0.55	15.29 ± 0.10
Sequence 3	12.24 ± 1.00	15.94 ± 0.43
Sequence 4	14.34 ± 0.40	15.49 ± 0.16



(a) $\mathcal{L}_{critic}$ curves



(b) Reward curves

FIGURE 9 $\mathcal{L}_{\text{critic}}$ curves and reward curves of agents with different reward functions in 2 × 3 grid under flowrate 600 vehicles per hour. The solid lines show the average $\mathcal{L}_{\text{critic}}$ and rewards, respectively. The shade shows the standard deviation over three independent runs.

of sequences. We can see that all the sequences have roughly the same and comparable performances. In conclusion, different orders of action sequences will not affect the learning processes.

5.3.3 | Calculation of reward-to-go

We will verify whether the proposed reward-to-go function G_t^k (Equation 11) can improve the stability and the speed of training. We compare agents with different reward functions (ours and the general reward-to-go) in 2 × 3 grid under 600 vehicles per hour inflow rate.

As shown in Figure 9, the loss of our critic $\mathcal{L}_{\text{critic}}$ is less than that of the general reward-to-go method (Figure 9a), which indicates that our reward function can improve critic's accuracy of predicting. Moreover, Figure 9b shows that the agent with our reward function can make the training faster (red curve rises quickly) and more stable.

6 | CONCLUSION

Here, we propose a centralized method based on DRL to control the grid. To overcome the problem of the curse of dimension, we propose a decomposition mechanism to reduce the search space, where the agent selects decisions incrementally based on LSTM network. This is a significant difference between our method and other DRL methods. The proposed method greatly reduces the action and the state space during the learning process and improves the flexibility of action selection. In addition, we propose a new training method for this mechanism. We have verified our method on both synthetic data and real-world data. The experiment results show that our method is easy to train. The comparisons with the traditional traffic lights control methods and DRL method suggest that our method can effectively control the traffic grid, and achieve high performance. Especially, it achieves the best queue length with an improvement of 21% over DRL method, in a complicated scenario with a high flowrate on synthetic data and the best travel time with an improvement of 9.35% over DRL method, on real-world data. Besides, we demonstrate that our method has good scalability and can control hundreds of intersections. In the future, we will try to adopt a more effective decomposition mechanism to extend our method to a larger network.

AUTHOR CONTRIBUTIONS

Jia Wu: Supervision; conceptualization; methodology; writing–reviewing and editing. Yunchuan Ran: Methodology; software; visualization; investigation; writing–original draft preparation. Yican Lou: Software; validation. Lingzhou Shu: Validation.

CONFLICT OF INTEREST STATEMENT

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available in traffic-signal-control at github. These data were derived from the following resources available in the public domain: https://traffic-signal-control.github.io/

ORCID

Jia Wu https://orcid.org/0000-0003-0599-0296

REFERENCES

1. Liang, X., Yan, T., Lee, J., Wang, G.: A distributed intersection management protocol for safety, efficiency, and driver's comfort. IEEE Internet Things J. 5, 1924–1935 (2018). https://doi.org/10.1109/JIOT.2018.2817459
2. Miller, A.J.: Settings for fixed-cycle traffic signals. Oper. Res. Q 14, 373–386 (1963)
3. Cools, S.B., Gershenson, C., D'Hooghe, B.: Self-organizing traffic lights: A realistic simulation. Advances in Applied Self-Organizing Systems (2006). https://doi.org/10.1007/978-1-84628-982-8_3

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

4. Isaac, P., Stéphane, L.: Adaptive look-ahead optimization of traffic signals. J. Intell. Transp. Syst. 4, 209–254 (1999). https://doi.org/10.1080/10248079908903749
5. Mikami, S., Kakazu, Y.: Genetic reinforcement learning for cooperative traffic signal control. In: Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence, vol. 1, pp. 223–228 (1994). https://doi.org/10.1109/ICEC.1994.350012
6. Medina, J.C., Hajbabaie, A., Benekohal, R.F.: Arterial traffic control using reinforcement learning agents and information from adjacent intersections in the state and reward structure. In: 13th International IEEE Conference on Intelligent Transportation Systems, pp. 525–530 (2010). https://doi.org/10.1109/ITSC.2010.5624977
7. LA, P., Bhatnagar, S.: Reinforcement learning with function approximation for traffic signal control. IEEE Trans. Intell. Transp. Syst. 12, 412–421 (2011). https://doi.org/10.1109/TITS.2010.2091408
8. Thorpe, T. L., Anderson, C. W.: Traffic light control using sarsa with three state representations. IBM Corporation Published (1996)
9. Abdulhai, B., Pringle, R., Karakoulas, G.J.: Reinforcement learning for true adaptive traffic signal control. J. Transp. Eng. 129, 278–285 (2003). https://doi.org/10.1061/(ASCE)0733-947X(2003)129:3(278)
10. Wen, K., Qu, S., Zhang, Y.: A stochastic adaptive control model for isolated intersections. In: 2007 IEEE International Conference on Robotics and Biomimetics (ROBIO), pp. 2256–2260 (2007). https://doi.org/10.1109/ROBIO.2007.4522521
11. Wiering, M.: Multi-agent reinforcement learning for traffic light control. In: Proceedings of the Seventeenth International Conference on Machine Learning, pp. 1151–1158. Morgan Kaufmann Publishers Inc., San Francisco, CA (2000)
12. Kuyer, L., Whiteson, S., Bakker, B., Vlassis, N.: Multiagent reinforcement learning for urban traffic control using coordination graphs. In: Daelemans, W., Goethals, B., Morik, K. (eds.), Machine Learning and Knowledge Discovery in Databases, pp. 656–671. Springer, Berlin, Heidelberg (2008)
13. , Abdulhai, B.: Adaptive traffic signal control with deep reinforcement learning and high dimensional sensory inputs: Case study and comprehensive sensitivity analyses. IEEE Trans. Intell. Transp. Syst. 23, 20021–20035 (2022). https://doi.org/10.1109/TITS.2022.3179893
14. Genders, W., Razavi, S.: Using a deep reinforcement learning agent for traffic signal control. arXiv:1611.01142 (2016). https://arxiv.org/abs/1611.01142
15. Gao, J., Shen, Y., Liu, J., Ito, M., Shiratori, N.: arXiv:1705.02755 (2017) https://arxiv.org/abs/1705.02755#:~:text=Adaptive%20Traffic%20Signal%20Control%3A%20Deep%20Reinforcement%20Learning%20Algorithm,be%20an%20effective%20method%20to%20reduce%20traffic%20congestion
16. Mousavi, S.S., Schukat, M., Howley, E.: Traffic light control using deep policy-gradient and value-function based reinforcement learning. IET Intel. Transport Syst. 11, 417–423 (2017)
17. Busoniu, L., Babuska, R., De Schutter, B.: Multi-agent reinforcement learning: A survey. In: 2006 9th International Conference on Control, Automation, Robotics and Vision, pp. 1–6 (2006). https://doi.org/10.1109/ICARCV.2006.345353
18. Choe, C., Baek, S., Woon, B., Kong, S.: Deep q learning with LSTM for traffic light control. In: 2018 24th Asia-Pacific Conference on Communications (APCC), pp. 331–336 (2018). https://doi.org/10.1109/APCC.2018.8633520
19. Richard, S.S., Andrew, G.B.: Reinforcement learning, second edition an introduction. Bradford Books, (2018) https://mitpress.mit.edu/9780262039246/reinforcement-learning/
20. Noaeen, M., Naik, A., Goodman, L., Crebo, J., Abrar, T., Abad, Z.S.H., Bazzan, A.L., Far, B.: Reinforcement learning in urban network traffic signal control: A systematic literature review. Expert Syst. Appl. 199, 116830 (2022). https://www.sciencedirect.com/science/article/pii/S0957417422002858,
21. Shu, L., Wu, J., Li, Z.: Hierarchical regional control for traffic grid signal optimization. In: 2019 IEEE Intelligent Transportation Systems Conference (ITSC), pp. 3547–3552 (2019). https://doi.org/10.1109/ITSC.2019.8917513
22. Zhang, R., Ishikawa, A., Wang, W., Striner, B., Tonguz, O. K.: Using reinforcement learning with partial vehicle detection for intelligent traffic signal control. IEEE Transactions on Intelligent Transportation Systems, 22(1), 404–415, (2021) https://doi.org/10.1109/TITS.2019.2958859
23. van der Pol, E., Oliehoek, F.A.: Coordinated deep reinforcement learners for traffic light control. NIPS'16 workshop on learning, inference and control of multi-agent systems (2016) https://sites.google.com/site/malicnips2016/papers
24. Shi, S., Chen, F.: Deep recurrent q-learning method for area traffic coordination control. Int. J. Appl. Math. Comput. Sci. 27, 1–11 (2018). https://doi.org/10.9734/JAMCS/2018/41281
25. Nishi, T., Otaki, K., Hayakawa, K., Yoshimura, T.: Traffic signal control based on reinforcement learning with graph convolutional neural nets. In: 21st International Conference on Intelligent Transportation Systems (ITSC), Maui, HI, USA, 2018, pp. 877–883 (2018) https://doi.org/10.1109/ITSC.2018.8569301
26. Riedmiller, M.: Neural fitted q iteration – first experiences with a data efficient neural reinforcement learning method. In: Proceedings of the 16th European Conference on Machine Learning, 317–328, Springer#x02010;Verlag, Porto, Portugal, (2005). https://doi.org/10.1007/11564096_32
27. Xu, M., Wu, J., Huang, L., Zhou, R., Wang, T., Hu, D.: Network-wide traffic signal control based on the discovery of critical nodes and deep reinforcement learning. J. Intell. Transp. Syst. 24(1), 1–10 (2019)
28. Xu, M., Wu, J., Liu, M., Xiao, Y., Wang, H., Hu, D.: Discovery of critical nodes in road networks through mining from vehicle trajectories. IEEE Trans. Intell. Transp. Syst. 20, 583–593 (2019). https://doi.org/10.1109/TITS.2018.2817282
29. Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., Zhang, W., Zhu, Y., Xu, K., Li, Z.: Colight: Learning network-level cooperation for traffic signal control. Proceedings of the 28th ACM International Conference on Information and Knowledge Management. Association for Computing Machinery, Beijing, China (2019). https://doi.org/10.1145/3357384.3357902
30. Zhong, T., Xu, Z., Zhou, F.: Probabilistic graph neural networks for traffic signal control. In: ICASSP 2021–2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 4085–4089 (2021). https://doi.org/10.1109/ICASSP39728.2021.9414829
31. Zhou, P., Braud, T., Alhilal, A., Hui, P., Kangasharju, J.: ERL: Edge based reinforcement learning for optimized urban traffic light control. In: 2019 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops), pp. 849–854 (2019). https://doi.org/10.1109/PERCOMW.2019.8730706
32. Hussain, A., Wang, T., Jiahua, C.: Optimizing traffic lights with multi-agent deep reinforcement learning and v2x communication. (2020). https://arxiv.org/abs/2002.09853
33. Ge, H., Song, Y., Wu, C., Ren, J., Tan, G.: Cooperative deep q-learning with q-value transfer for multi-intersection signal control. IEEE Access 7, 40797–40809 (2019). https://doi.org/10.1109/ACCESS.2019.2907618
34. Chu, T., Wang, J., Codecà, L., Li, Z.: Multi-agent deep reinforcement learning for large-scale traffic signal control. IEEE Trans. Intell. Transp. Syst. 21, 1086–1095 (2020). https://doi.org/10.1109/TITS.2019.2901791
35. Zhang, L., Wu, Q., Shen, J., Lü, L., Du, B., Wu, J.: Expression might be enough: representing pressure and demand for reinforcement learning based traffic signal control. In: Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., Sabato, S. (eds.), Proceedings of the 39th International Conference on Machine Learning, pp. 26645–26654. PMLR, (2022)
36. Zhao, W., Ye, Y., Ding, J., Wang, T., Wei, T., Chen, M.: IPDAlight: Intensity- and phase duration-aware traffic signal control based on reinforcement learning. J. Syst. Archit. 123, 102374 (2022). https://www.sciencedirect.com/science/article/pii/S1383762121002587
37. Wu, Q., Wu, J., Shen, J., Du, B., Telikani, A., Fahmideh, M., Liang, C.: Distributed agent-based deep reinforcement learning for large scale traffic signal control. Knowl. Based Syst. 241, 108304 (2022). https://doi.org/10.1016/j.knosys.2022.108304

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

38. Meess, H., Gerner, J., Hein, D., Schmidtner, S., Elger, G.: Reinforcement learning for traffic signal control optimization: A concept for real-world implementation. In: Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems. International Foundation for Autonomous Agents and Multiagent Systems, pp. 1699–1701. Richland, SC (2022)
39. Wang, H., Yuan, Y., Yang, X.T., Zhao, T., Liu, Y.: Deep q learning-based traffic signal control algorithms: Model development and evaluation with field data. J. Intell. Transp. Syst. 1–21 Taylor & Francis, (2022). https://doi.org/10.1080/15472450.2021.2023016
40. Zhu, R., Wu, S., Li, L., Lv, P., Xu, M.: Context-aware multiagent broad reinforcement learning for mixed pedestrian-vehicle adaptive traffic light control. IEEE Internet Things J. 9, 19694–19705 (2022). https://doi.org/10.1109/JIOT.2022.3167029
41. Schmidhuber, J.: Reinforcement learning upside down: Don't predict rewards – just map them to actions. arXiv:1912.02875 (2020) https://arxiv.org/abs/1912.02875
42. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv:1707.06347 (2017) https://arxiv.org/abs/1707.06347
43. Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P.: High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438 (2018) https://arxiv.org/abs/1506.02438
44. Zheng, G., Xiong, Y., Zang, X., Feng, J., Wei, H., Zhang, H., Li, Y., Xu, K., Li, Z.: Learning phase competition for traffic signal control. In: Proceedings of the 28th ACM International Conference on Information and Knowledge Management, pp. 1963–1972 (2019)
45. Wei, H., Zheng, G., Gayah, V., Li, Z.: A survey on traffic signal control methods. arXiv:1904.08117 (2019) https://arxiv.org/abs/1904.08117
46. Wei, H., Zheng, G., Yao, H., Li, Z.: Intellilight: A reinforcement learning approach for intelligent traffic light control. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2496–2505. Association for Computing Machinery, New York (2018). https://doi.org/10.1145/3219819.3220096
47. Arel, I., Liu, C., Urbanik, T., Kohls, A.G.: Reinforcement learning-based multi-agent system for network traffic signal control. Intelligent Transport Systems Iet 4, 128–135 (2010)
48. Gordon, R.L., Warren Tighe, P.E.: Traffic Control Systems Handbook. United States. Federal Highway Administration. Office of Transportation Management, (2005) https://ops.fhwa.dot.gov/publications/fhwahop06006/index.htm

How to cite this article: Wu, J., Ran, Y., Lou, Y., Shu, L.: CenLight: Centralized traffic grid signal optimization via action and state decomposition. IET Intell. Transp. Syst. 17, 1247–1261 (2023).
https://doi.org/10.1049/itr2.12354

17519578, 2023, 6, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12354, Wiley Online Library on [01/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License